

The Impact of Multithreading and Multiprocessing on Deep Learning Performance



Dataset: EuroSAT

Problem: Image Classification

Kelompok:

1. Deas Putra Fajar Ramadhan
2. Kelvin Nabil Anshary
3. Samuel Parmonangan Sianturi

PROBLEM & OBJECTIVE



Problem

Deep Learning membutuhkan proses data dalam jumlah besar. Cara data diproses oleh CPU sebelum dikirim ke GPU dapat memengaruhi performa training.

Objective

Eksperimen dilakukan untuk mengetahui:

- Pengaruh multithreading terhadap performa.
- Pengaruh multiprocessing terhadap performa.
- Pengaruh jumlah worker terhadap throughput dan execution time.
- Bottleneck utama pada pipeline Deep Learning.

Main Question

“Does increasing the number of threads or processes always improve Deep Learning performance?”

EuroSAT

Dataset citra Sentinel-2 untuk land-use and land-cover classification.

Dataset Information

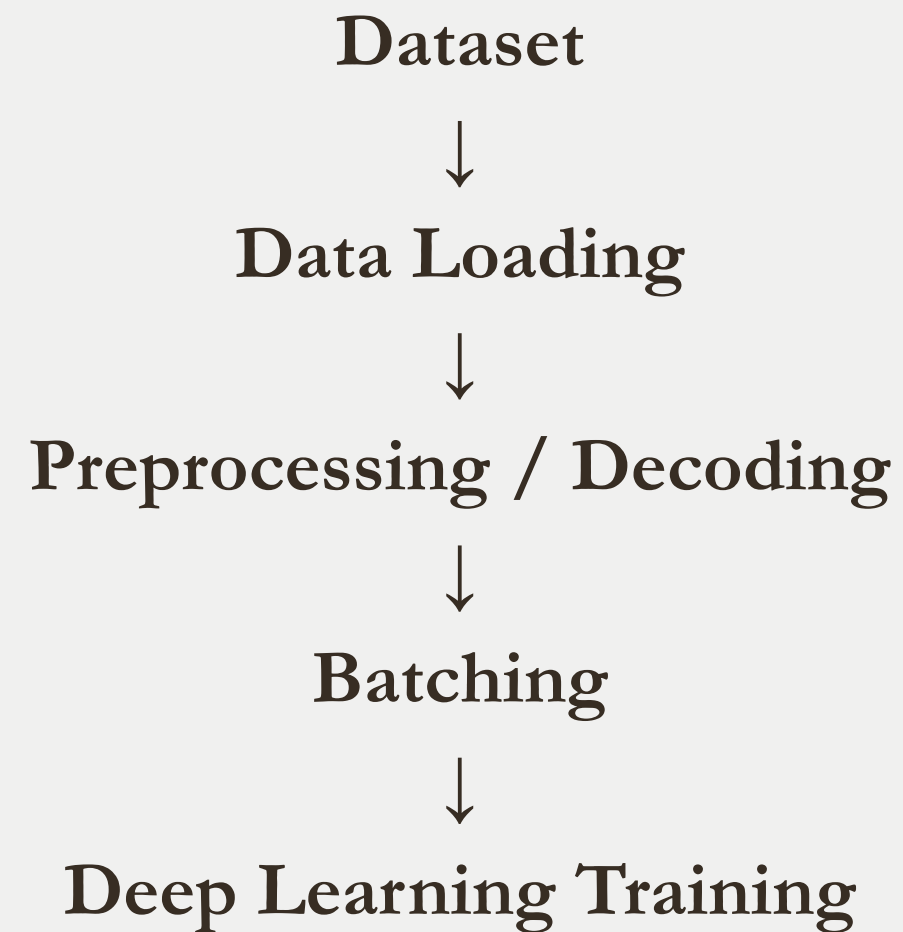
Characteristic	Value
Images	27,000
Classes	10
Image Size	64 x 64
Resolution	10 m/pixel
RGB	3 bands
Multispectral	13 bands

Dataset Variants

RGB	Multispectral
JPEG	GeoTIFF
94.7 MB	2.07 GB
3channels	13 channels

- Yang dipakai:
- Layer A: RGB + Multispectral
 - Layer B: Multispectral

Pipeline yang diuji



Fokus eksperimen:

Bagaimana proses data loading dan preprocessing memengaruhi training performance?

Three Processing Methods

Method	Implementation
Sequential	Single process, no parallel worker
Multithreading	Multiple threads
Multiprocessing	Multiple processes / workers



Worker Configurations

Multithreading: 2–16 threads

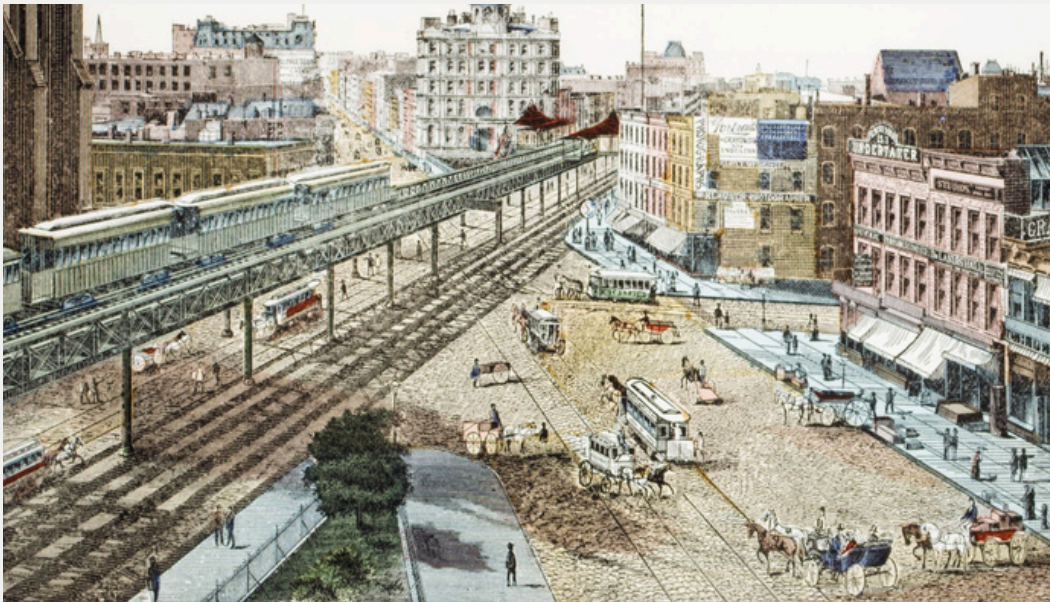
Multiprocessing: 1–16 workers

Benchmark

- Warm-up dilakukan untuk mengurangi pengaruh OS page cache.
- CUDA synchronization digunakan untuk pengukuran GPU.
- Thread PyTorch diatur secara eksplisit.
- Hasil disimpan setelah setiap konfigurasi.

Experimental Environment

	Sandbox	Kaggle
CPU	Xeon Platinum 8488C	Xeon @ 2.00 GHz
Physical Core	4	2
Logical Core	8	4
RAM	32GB	31GB
GPU	None	T4 x2



- Software
- Linux
 - Python
 - PyTorch
 - CUDA

Why Multiple Tiers?

Perbedaan jumlah core digunakan untuk melihat bagaimana performa parallelism berubah berdasarkan hardware.

Metrics

Eksperimen mengukur:

- Execution / Training Time
- Time per Epoch
- Data Loading Time
- Throughput (images/sec)
- CPU Utilization
- RAM Usage
- GPU Utilization

Speedup

$$Speedup = \frac{T_{sequential}}{T_{parallel}}$$

Parallel Efficiency

Digunakan untuk melihat seberapa efektif tambahan worker menghasilkan speedup.



Layer A - Preprocessing

Multithreading Performance

[GRAFIK: Execution Time vs Number of Threads]

GRAFIK: Speedup vs Number of Threads]

Key Results

- MS $\times 2 \rightarrow 1.08\times$
- MS $\times 4 \rightarrow 1.02\times$
- MS $\times 8 \rightarrow 0.96\times$
- RGB $\times 8 \rightarrow 0.91\times$

Finding

“Increasing the number of threads did not consistently improve performance.”

Pada konfigurasi tertentu, performa justru menurun.

Penyebab utama: Python GIL membatasi keuntungan multithreading untuk workload CPU-bound.

Multiprocessing Performance

[GRAFIK: Execution Time vs Number of Workers]

[GRAFIK: Throughput vs Number of Workers]

Key Results

- MS $\times 4 \rightarrow 1.84\times$ speedup
- RGB $\times 8 \rightarrow 2.42\times$ speedup

Finding

“Multiprocessing memberikan peningkatan performa yang lebih jelas dibandingkan multithreading pada preprocessing.”

Namun, peningkatan tidak terus bertambah secara linear.

.

Effect of Workload Size

Pada 8 workers:

Dataset Size	Speedup
500	1.52x
2,000	1.92x
8,000	2.41x

[GRAFIK: Workload Size vs Speedup]

Finding

“Semakin besar workload, overhead multiprocessing menjadi semakin kecil dibandingkan pekerjaan yang dilakukan.”

Sehingga speedup menjadi lebih terlihat pada workload yang lebih besar.

.

Process Communication Overhead

Beban CPU dibuat identik, tetapi data yang dikembalikan berbeda.

Payload	Speedup: 8 Workers
Array	2.13x
Scalar	4.29x

Pada 2 workers:

- Array → 0.99×
- Scalar → 1.89×

Finding

“IPC/data transfer merupakan bottleneck utama, bukan CPU decoding.”

[Gambar/diagram: Worker → IPC → Main Process]

Layer B - Deep Learning Training

GPU Training

[GRAFIK: GPU Utilization vs Number of Workers]

[GRAFIK: Training Throughput vs Number of Workers]

Key Results

Configuration	GPU Idle	Throughput
workers=0	37.3%	Baseline
workers=1	2.8%	1.50x

Finding

Dengan workers=0, GPU harus menunggu data dari CPU.

Setelah worker ditambahkan:

GPU idle time turun **37.3% → 2.8%**

.

Where Is the Bottleneck?

Sandbox

Data loading hanya **1.2%** dari waktu step.

- Compute-bound
- Amdahl limit: **1.01×**

Kaggle

Data loading mencapai **43.7%**.

- Data-bound
- Amdahl limit: **1.77×**

[Gambar: CPU → Data Loading → GPU dengan bottleneck ditandai]

Key Finding

Bottleneck berbeda berdasarkan hardware dan workload.

Theoretical Limit vs Actual Performance

Environment	Data Loading	Amdahl Limit
Sandbox	1.2%	1.01x
Kaggle	43.7%	1.77x

Interpretation

Semakin besar bagian pipeline yang dapat diparalelkan, semakin besar potensi speedup.

Namun:

Parallelism cannot speed up the serial portion of a workload.

Final Analysis

Sequential vs Multithreading vs Multiprocessing

[GRAFIK UTAMA: Execution Time vs Processing Method]

Method	Main Result
Sequential	Baseline
Multithreading	Limited / sometimes slower
Multiprocessing	Significant improvement for CPU-bound preprocessing

Overall Finding
Multiprocessing memberikan keuntungan terbesar ketika preprocessing menjadi bottleneck.

ANSWERING THE ASSIGNMENT QUESTIONS

1. Is multiprocessing always faster than sequential?

No.

Performance bergantung pada workload, hardware, dan overhead.

2. more workers always faster?

No.

Terlalu banyak worker dapat meningkatkan overhead.

3. Does multithreading improve performance?

Not consistently.

GIL dapat membatasi CPU-bound Python workload.

4. What is the main bottleneck?

Bergantung pada eksperimen:

- CPU/data processing
- IPC
- GPU/data dependency

5. Is GPU starvation observed?

Yes.

workers=0 → GPU idle **37.3%**.

1. Multithreading tidak selalu meningkatkan performa karena GIL dan overhead.
2. Multiprocessing dapat meningkatkan performa pada workload CPU-bound.
3. Lebih banyak worker tidak selalu berarti lebih cepat.
4. Bottleneck harus diidentifikasi sebelum menentukan strategi parallelism.
5. Pada training GPU, worker dapat membantu ketika GPU harus menunggu data.

Main Conclusion

| The assumption “more threads/processes always means faster Deep Learning” is rejected.

Performa terbaik bergantung pada **workload, hardware, bottleneck, dan jumlah worker.**

Source Code & Reproducibility

LINK GITHUB

| All experiments are designed to be reproducible.

THANK YOU